CSE 209 — Computer Architecture

Chapter 5 · Large and Fast: Exploiting Memory Hierarchy

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Rifat Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, bit layouts and hand-drawn datapath sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — memory-hierarchy questions by year, most-repeated first
Lesson 19 — Locality, Hierarchy and Memory Technology — hit/miss terms, SRAM/DRAM/flash/disk, disk access time
Lesson 20 — Direct-Mapped Caches and Traces — block address, index, tag, valid bit, hit/miss trace
Lesson 21 — Address Subdivision and Total Cache Bits — offset/index/tag, total bits, overhead
Lesson 22 — Set-Associative Caches and LRU Traces — DM vs n-way vs fully associative, LRU tables
Lesson 23 — Writes, Block Size and Miss Types — write-through/back, write buffer, allocate, 3 Cs
Lesson 24 — Cache Performance: CPI, AMAT, Multilevel — memory stalls, AMAT, L2/L3 speedup
Lesson 25 — Software and the Cache — blocking, radix vs quicksort, byte vs word pitfall
Lesson 26 — Virtual Memory and Page Tables — translation, page faults, table sizes, multilevel, inverted
Lesson 27 — TLBs and the TLB–Page Table–Cache Path — TLB miss vs page fault, traces, flowchart, scenarios
Lesson 28 — Cache Coherence — snooping, directory, invalidation example, consistency
Quick Notes — All Lessons — merged revision notes + key diagrams

NOTE Slide numbers refer to **Chapter 5** (93 slides). TF questions are labelled by **session and course code**. Practice provenance: **Actual** = taken from a TF paper; **Adapted** = TF question with changed details; **Constructed** = new question in TF style. Every trace and number in the solutions was checked with a small simulation. A few topics the papers ask but the slides only touch (total cache bits, multilevel/inverted page tables) follow the textbook and are marked.

Exam Map — What the TF Papers Ask

Memory hierarchy is the **largest numerical block** in every paper. The newest five sessions are the priority.

Session (paper)	Memory-hierarchy questions
2023-24 (CSE 209)	Q2(a) **L2 then L3 speedup** (CPI 1, 4 GHz, 200 ns, 5% → 2% → 1%) · Q2(b) **direct-mapped byte-address trace** (tag 31–10, index 9–5, offset 4–0): block size, entries, replacements, hit ratio, final state · Q2(c) larger blocks, early restart / critical word first, **write buffer** · Q3(a) TLB vs data cache + **TLB–cache flowchart** · Q3(b) **TLB + page table trace** with page faults · Q3(c) **coherence + snooping**
2022-23 (CSE 209)	Q9(a) **P1 (L1+L2) vs P2 (L1+L2+L3)** effective CPI, required L2 miss rate · Q9(b) **total size of a 4-way cache** (48-bit address, 64 KB, 16-word blocks, valid/dirty/protection bits) · Q10(a) **page-table memory**: single-level, three-level (best/average/worst), inverted · Q10(b) **TLB + data-cache block diagram** with labels · Q10(c) TLB/cache on context switch · Q12(d) radix vs quicksort graphs
2022-23 (CSE 305)	Q2(a) **DM vs 2-way LRU** table (32, 36, 35, 32, 48, 32, 51, 35), hit ratio · Q2(b) **total bits, 8-way**, 4K blocks · Q3(a) **two-level CPI** (CPI 2, 0.5 ns, 150 ns) · Q3(b) virtual vs physical address · Q3(c) page-table entries (64-bit VA, 8 KB page)
2021-22 (CSE 305)	Q3(a) **L2 / L3 speedup** (4% → 1% → 0.5%) · Q3(b) page table diagram · Q3(c) **TLB / page table / cache hit-miss scenarios** · Q3(d) snooping · Q4(a) **FA, 2-way, DM misses** (1, 3, 2, 4, 5, 1, 3, 1, 3, 5, 3, 2)
2020-21 (CSE 305)	Q6 **address subdivision** 12/8/10/2 bits, 2-way: blocks, index of an address, replacement probability · Q7 **four-level hierarchy** probabilities, expected time, split I/D miss rate · Q8 **approximate-LRU page replacement** with reference bits
2019-20 (CSE 305)	Q6(a) **DM / 2-way / 4-way** table (0, 12, 16, 18, 0) · Q6(b) **tag bits, 8-way**, 4K blocks · Q7(a) **total bits, DM 64 KB, 8-word blocks** · Q7(b) purpose of TLB tag bits · Q7(c) **how a TLB miss is handled**
2018-19	Q3(a) TLB–cache–VM flowchart · Q3(b) **page-table size**, inverted table, TLB mapping (40-bit VA, 16 KB pages, 36-bit PA) · Q3(c) 16-entry TLB vs 2 MB working set · Q4(a) three processors with different caches · Q4(b) **overhead DM / 2-way / FA** (256 KB, 64-word blocks), cache address of lw
2017-18	Q3(a) **3 Cs + disk read time** · Q3(b) locality, **write-through/back, write miss** · Q3(c) page table · Q3(d) **I/D-cache CPI** (3%, 5%, 40%, 200 cycles) · Q4(a) L2/L3 speedup · Q4(b) **three caches LRU** (1, 3, 5, 1, 3, 1, 3, 5, 3, 1, 5, 3) · Q4(c) why TLB · Q4(d) TLB–cache **write** flowchart
Older papers	DM byte trace (2015-16, same as 2023-24) · three caches LRU (2015-16, 2014-15) · I/D CPI (2015-16, 2014-15) · write-through/back (2015-16) · page table sizes (2013-14, 2014-15) · tag-field size, 2-way hits, coherence conditions, TLB miss procedure (2013-14) · disk time, compulsory vs conflict misses, sets for n-way, L2 options CPI (2012-13)

EXAM FOCUS **Most repeated, in order:** (1) **multilevel-cache CPI / speedup** (almost every year) · (2) **cache traces** — direct-mapped byte addresses and DM/2-way/FA LRU tables · (3) **address bits and total cache bits / overhead** · (4) **TLB + page-table trace** and the **TLB / page table / cache scenario** table · (5) **page-table size** (single, multilevel, inverted) · (6) **TLB–cache flowchart / block diagram** · (7) write policies + write buffer · (8) **snooping coherence** example.

Lesson 19 — Locality, Hierarchy and Memory Technology

Slides: Chapter 5, 2–16

EXAM FOCUS Short theory: locality and its role in the hierarchy (2017-18 Q3(b), 2014-15, 2015-16), memory technologies, NAND vs NOR flash (2012-13). Numerical: **average disk read time** (2017-18 Q3(a), 2012-13 Q5(a)).

1. Principle of Locality (slide 2)

Programs access a **small proportion of their address space** at any time.

Kind	Meaning	Example
Temporal	Items accessed recently are likely to be accessed again soon	Instructions in a loop, induction variables
Spatial	Items **near** those accessed recently are likely to be accessed soon	Sequential instruction access, array data

Role in the hierarchy (slide 3): store everything on disk; copy recently accessed **(temporal)** and nearby **(spatial)** items from disk into smaller DRAM main memory; copy more recently accessed and nearby items from DRAM into an even smaller SRAM **cache** attached to the CPU. Because of locality most accesses hit in the small, fast levels, so the system looks **as fast as SRAM and as large as disk**.

2. Hierarchy Terms (slide 4)

- **Block (line):** unit of copying between levels; may be several words.
- **Hit:** access satisfied by the upper level. **Hit ratio** = hits / accesses.
- **Miss:** block copied from the lower level. **Miss ratio** = misses / accesses = 1 − hit ratio.
- **Miss penalty:** time to bring the block in (then the data is supplied from the upper level).

3. Memory Technologies (slides 5–12)

Technology	Access time	$ per GB	Notes
SRAM	0.5–2.5 ns	$2000–$5000	Caches
DRAM	50–70 ns	$20–$75	Main memory; charge in a capacitor, one transistor, **must be refreshed** (read and write back a whole row)
Flash	100–1000× faster than disk	between disk and DRAM	Nonvolatile; wears out after 1000s of writes → **wear levelling**
Magnetic disk	5–20 ms	$0.20–$2	Nonvolatile, rotating

Ideal memory: the access time of SRAM with the capacity and cost/GB of disk.

DRAM bandwidth tricks: row buffer (several words read/refreshed in parallel), **burst mode** / synchronous DRAM (consecutive words without new addresses), **DDR** (transfer on both clock edges), QDR (separate DDR inputs and outputs), **banking** (simultaneous access to several banks).

	NOR flash	NAND flash
Bit cell	Like a NOR gate	Like a NAND gate
Access	**Random** read/write	**Block-at-a-time**, denser, cheaper per GB
Used for	Instruction memory in embedded systems	USB keys, media storage

4. Worked Example: Disk Access Time (slides 14–15)

Access to a sector = **queuing delay + seek + rotational latency + transfer + controller overhead**.

Given: 512 B sector, 15,000 rpm, 4 ms average seek, 100 MB/s transfer, 0.2 ms controller, idle disk.

Rotational latency = ½ rotation = 0.5 / (15,000 / 60) s = **2 ms**
Transfer = 512 B / 100 MB/s = **0.005 ms**
Average read time = 4 + 2 + 0.005 + 0.2 = **6.2 ms**

If the actual average seek is 1 ms (locality, OS scheduling): **3.2 ms**.

> **MEMORIZE** Rotational latency is **half a rotation**: 0.5 / (rpm / 60) seconds. The quoted average seek time is usually pessimistic.

Practice — Lesson 19

Actual — 2017-18 Q3(a) (CSE 305, scanned page 34).

(a) What are the three sources of cache misses? *(answered in Lesson 23)* — An ideal disk has: sector size 1 MB, rotational speed 10,000 rpm, average seek 5 ms, transfer rate 100 MB/s, controller overhead 0.5 ms. What is the average read time? If the average seek time is reduced to 2 ms, how much improvement is possible?

(b) What do you mean by spatial and temporal locality? *(2017-18 Q3(b))*

✓ Solution

(a)

Rotational latency = 0.5 / (10,000 / 60) = 3 ms Transfer = 1 MB / (100 MB/s) = 10 ms
Average read time = 5 + 3 + 10 + 0.5 = **18.5 ms**
With 2 ms seek: 2 + 3 + 10 + 0.5 = **15.5 ms** → 18.5 / 15.5 = **1.19× faster** (16.2 % less time)

(b) Temporal locality: a recently accessed item is likely to be accessed again soon (loop instructions, loop counters). Spatial locality: items close to a recently accessed item are likely to be accessed soon (sequential instructions, array elements). Caches exploit temporal locality by **keeping** recently used blocks and spatial locality by **bringing whole blocks**.

Lesson 20 — Direct-Mapped Caches and Traces

Slides: Chapter 5, 17–27

EXAM FOCUS **2023-24 Q2(b)** (identical to 2015-16 Q4(b)): block size, number of entries, replaced blocks, hit ratio and final cache state for byte addresses. Direct-mapped columns also appear in every LRU-table question (Lesson 22).

1. Where Does a Block Go? (slides 18–19)

Direct mapped: each memory block has **only one** possible cache location.

> **Cache index = (Block address) modulo (Number of blocks in cache)**

With a power-of-2 number of blocks, the index is simply the **low-order bits** of the block address.

- **Tag:** the remaining high-order bits, stored with the data, to tell **which** block is in that location.
- **Valid bit:** 1 = present, 0 = not present; all 0 at power-on.

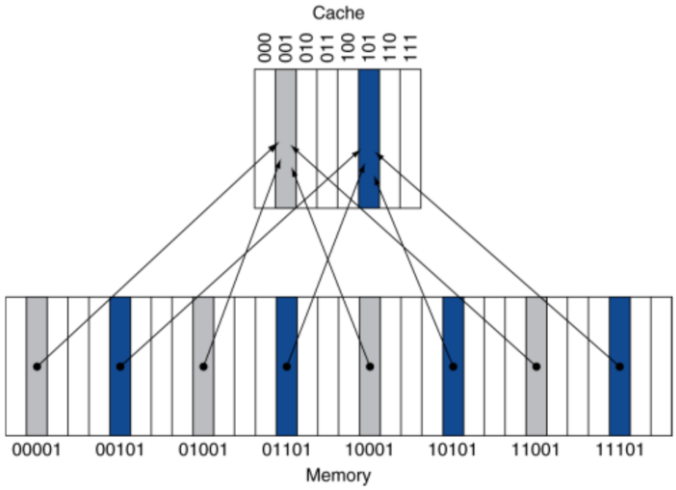


Figure 20.1 — Direct-mapped cache with 8 entries: memory blocks map by their low 3 bits (slide 18)

2. Worked Example: 8 Blocks, 1 Word per Block (slides 20–25)

Word addresses 22, 26, 22, 26, 16, 3, 16, 18. Index = low 3 bits, tag = the rest.

Word addr	Binary	Index	Result	Notes
22	10 110	110	Miss	compulsory; tag 10 stored
26	11 010	010	Miss	compulsory
22	10 110	110	**Hit**	
26	11 010	010	**Hit**	
16	10 000	000	Miss	
3	00 011	011	Miss	
16	10 000	000	**Hit**	
18	10 010	010	Miss	**replaces Mem[11010]** (tag 11 → 10)

Final state: 000 → tag 10 Mem[10000] · 010 → tag 10 Mem[10010] · 011 → tag 00 Mem[00011] · 110 → tag 10 Mem[10110]; others invalid.

3. Byte Addresses and Larger Blocks (slides 26–27)

With **byte** addresses and blocks of B bytes:

Block address = ⌊ byte address / B ⌋ Index = Block address mod (#blocks) Tag = ⌊ Block address / #blocks ⌋

Slide 27 example: 64 blocks, 16 bytes/block. Address 1200 → block address ⌊1200/16⌋ = **75** → index 75 mod 64 = **11**. Fields: tag 22 bits | index 6 bits | offset 4 bits.

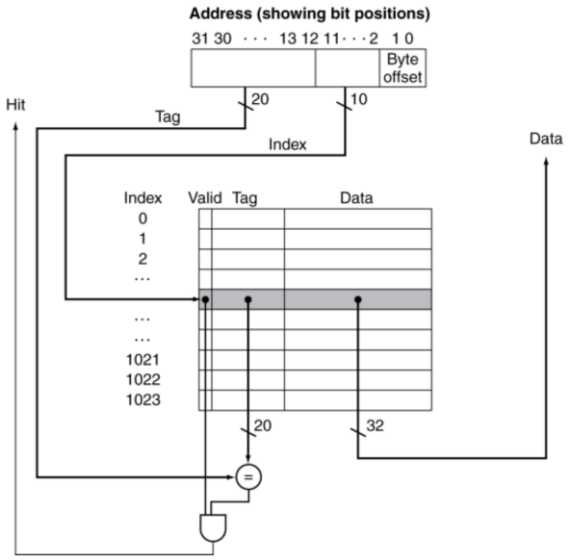


Figure 20.2 — Address subdivision: 1024 one-word blocks (slide 26)

Exam Approach — Direct-Mapped Trace

1. From the field boundaries get **offset bits → block size**, **index bits → number of entries**.
2. For each address: block address = addr ÷ block size (drop the remainder); index = block address mod entries; tag = addr ÷ (block size × entries).
3. **Hit** only if that index is valid **and** has the same tag; otherwise miss, and if the entry was valid it is a **replacement**.
4. Keep a running table of index → (tag, block).

TRAP

- Two different byte addresses in the **same block** hit (0, 4, 16 are all in block 0 when blocks are 32 bytes).
- A miss into an **empty** entry is not a replacement.
- Block size in **words** = bytes ÷ 4.

Practice — Lesson 20

Actual — 2023-24 Q2(b) (also 2015-16 Q4(b)).

A direct-mapped cache with a 32-bit address uses: **Tag 31–10, Index 9–5, Offset 4–0**. Starting from power on, the byte-addressed references are: **0, 4, 16, 132, 232, 160, 1024, 30, 140, 3100, 180, 2180**.

(i) Block size in words? (ii) Number of entries? (iii) How many blocks are replaced? (iv) Hit ratio? (v) Final state as ⟨index, tag, data⟩.

✓ Solution

(i) Offset = 5 bits → 32 bytes = **8 words**. **(ii)** Index = 5 bits → **32 entries**.

Block address = ⌊addr / 32⌋, index = block mod 32, tag = ⌊addr / 1024⌋:

Address	Block addr	Index	Tag	Result
0	0	0	0	Miss
4	0	0	0	**Hit**
16	0	0	0	**Hit**
132	4	4	0	Miss
232	7	7	0	Miss
160	5	5	0	Miss
1024	32	0	1	Miss — **replaces** block 0 (tag 0)
30	0	0	0	Miss — **replaces** block 32 (tag 1)
140	4	4	0	**Hit**
3100	96	0	3	Miss — **replaces** block 0 (tag 0)
180	5	5	0	**Hit**
2180	68	4	2	Miss — **replaces** block 4 (tag 0)

(iii) 4 blocks replaced. (iv) Hits = 4 of 12 → **hit ratio = 1/3 ≈ 33.3 %**.

(v) Final valid entries:

Index	Tag	Data
0	3	Mem[3072–3103] (block 96)
4	2	Mem[2176–2207] (block 68)
5	0	Mem[160–191] (block 5)
7	0	Mem[224–255] (block 7)

Lesson 21 — Address Subdivision and Total Cache Bits

Slides: Chapter 5, 26–27 and 44 · total-bits method follows the textbook (§5.3–5.4)

EXAM FOCUS **Total bits / tag bits / overhead** in almost every paper: 2022-23 (CSE 209) Q9(b), 2022-23 (CSE 305) Q2(b), 2019-20 Q6(b) and Q7(a), 2018-19 Q4(b), 2020-21 Q6, 2013-14, 2012-13.

1. The Three Fields

For a byte address of **A** bits, block size **B bytes**, **S sets** (S = blocks for direct mapped, blocks / n for n-way, 1 for fully associative):

Offset bits = $\log_2 B$	**Index bits = $\log_2 S$**	**Tag bits = A − index − offset**

```
A-1                                                              0
┌──────────────────────────────┬──────────────┬──────────────────┐
│             Tag              │    Index     │      Offset      │
│                              │ (selects set)│ (word + byte)    │
└──────────────────────────────┴──────────────┴──────────────────┘
```

- Offset = **word offset + byte offset** (2 bits for 4-byte words). If a question says "byte offset = 2 bits", the rest is the word offset.
- More associativity → fewer sets → **index shrinks by 1 bit and tag grows by 1 bit** per doubling.
- Fully associative: **no index**; tag = A − offset.

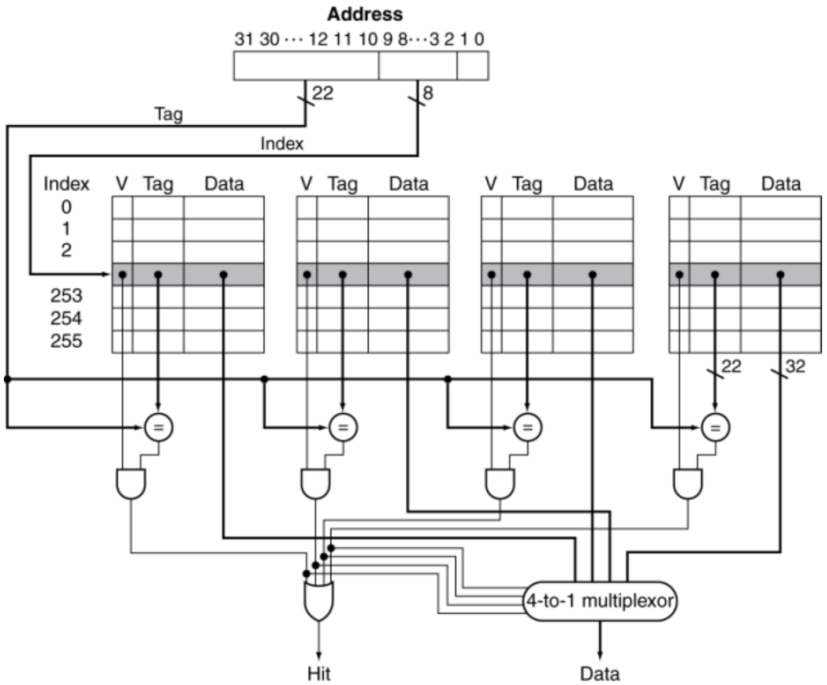


Figure 21.1 — 4-way set-associative cache: 256 sets, 4 comparators and a 4-to-1 mux (slide 44)

2. Worked Example: Total Bits (textbook)

Direct-mapped, **16 KiB of data, 4-word blocks, 32-bit address**.

1. Block = 16 bytes → offset **4** bits.
2. Blocks = 16 KiB / 16 B = 1024 → index **10** bits.
3. Tag = 32 − 10 − 4 = **18** bits.
4. Bits per line = data 4 × 32 = 128 + tag 18 + valid 1 = **147**.

Total = 1024 × 147 = **147 Kibits** ≈ 18.4 KiB (1.15 × the data size)

MEMORIZE **Bits per line = data bits + tag + valid (+ dirty + protection/other bits given) ; Total = number of blocks × bits per line.** "Overhead" = everything except the data.

Exam Approach

1. Convert everything to **bytes** and **powers of 2**.
2. Blocks = data size ÷ block size; sets = blocks ÷ associativity.
3. Fields as above; then per-line bits; then total.
4. Give the answer in bits, then in KiB if useful (÷ 8 ÷ 1024).

TRAP

- Use the **data** size for the number of blocks, never the total size.
- Count extra bits the question lists (dirty, protection, LRU, bookkeeping) — and **only** those.
- "lb support" / byte-addressable means the offset includes the **byte** bits.

Practice — Lesson 21

(a) Actual — 2019-20 Q7(a). (b) Actual — 2019-20 Q6(b) / 2022-23 (CSE 305) Q2(b). (c) Actual — 2022-23 (CSE 209) Q9(b). (d) Actual — 2018-19 Q4(b). (e) Actual — 2020-21 Q6.

(a) How many total bits are required for a direct-mapped cache with 64 KB of data and 8-word blocks, assuming a 32-bit address?

(b) A cache has 4K blocks, a 4-word block size and a 32-bit address. Find the total number of tag bits, and the total bits, for an **eight-way** set-associative cache.

(c) Determine the total size of a 4-way set-associative cache: address 48 bits, 64 KB data, 16-word blocks, with tag, valid, dirty and protection bits; the hardware supports lb (load byte).

(d) 32-bit address, byte-addressable, 256 KB cache, 64-word blocks, random replacement, write-through. What is the overhead if the cache is (i) direct mapped, (ii) 2-way, (iii) fully associative? With direct mapping and $2 = 1000, where in the cache does lw $1, 18($2) look?

(e) Address fields: tag 31–20 (12 bits), index 19–12 (8 bits), word offset 11–2 (10 bits), byte offset 1–0; 2-way set-associative LRU cache. (a) How many distinct blocks can the cache hold? (b) Which index does decimal address 24092022 map to? (c) If an index is full, what is the probability that the next request replaces one of its blocks, assuming all addresses are equally likely?

✓ Solution

(a) Block = 32 B → offset 5; blocks = 64 KB / 32 B = 2048 → index 11; tag = 32 − 11 − 5 = 16. Per line = 256 + 16 + 1 = 273.

Total = 2048 × 273 = **559,104 bits** ≈ 68.25 KiB

(b) Sets = 4096 / 8 = 512 → index 9; offset 4; tag = 32 − 9 − 4 = **19**.

Tag bits = 19 × 4096 = **77,824 bits** (76 Kbits) Total = 4096 × (128 + 19 + 1) = **606,208 bits** (592 Kbits)

(c) Block = 64 B → offset 6 (4 word + 2 byte); blocks = 64 KB / 64 B = 1024; sets = 256 → index 8; tag = 48 − 8 − 6 = **34**. Per line = 34 + 1 + 1 + 1 + 512 = **549 bits**.

Total = 1024 × 549 = **562,176 bits** = 70,272 bytes ≈ 68.6 KiB

(d) Block = 256 B → offset 8; blocks = 256 KB / 256 B = 1024. Write-through → no dirty bit; random → no LRU bits. Overhead per block = tag + valid.

Organization	Index	Tag	Per block	Overhead
Direct mapped	10	14	15	**15,360 bits**
2-way (512 sets)	9	15	16	**16,384 bits**
Fully associative	0	24	25	**25,600 bits**

Address = 1000 + 18 = **1018** → block address ⌊1018/256⌋ = 3 → **index 3, byte offset 250, tag 0**.

(e) (a) 2^8 = 256 sets × 2 ways = **512 blocks** (each 2^{12} B = 4 KB). (b) 24092022 = 0x016F9D76 → index = bits 19–12 = 0xF9 = **249** (tag = 0x016 = 22). (c) The request must map to this index (probability 1/256) and miss both stored tags ((4096 − 2)/4096):

$$P = (1/256) \times (4094/4096) \approx \mathbf{0.0039}\ (0.39\ \%)$$

Lesson 22 — Set-Associative Caches and LRU Traces

Slides: Chapter 5, 38–45 and 74–76

EXAM FOCUS **LRU tables for several organizations** — one of the most repeated questions: 2022-23 (CSE 305) Q2(a), 2021-22 Q4(a), 2019-20 Q6(a), 2017-18 Q4(b), 2015-16, 2014-15, 2013-14.

1. Three Placement Policies (slides 38–40, 74–75)

Organization	Where can a block go?	Set / index	Tag comparisons
Direct mapped (1-way)	Exactly one entry	Block mod #blocks	1
n-way set associative	Any of the n entries in one set	Block mod #sets	n (in parallel)
Fully associative	Any entry	none — search all	#entries (one comparator each — expensive)



Figure 22.1 — Locating block 12 in an 8-block cache: direct mapped, 2-way, fully associative (slide 39)

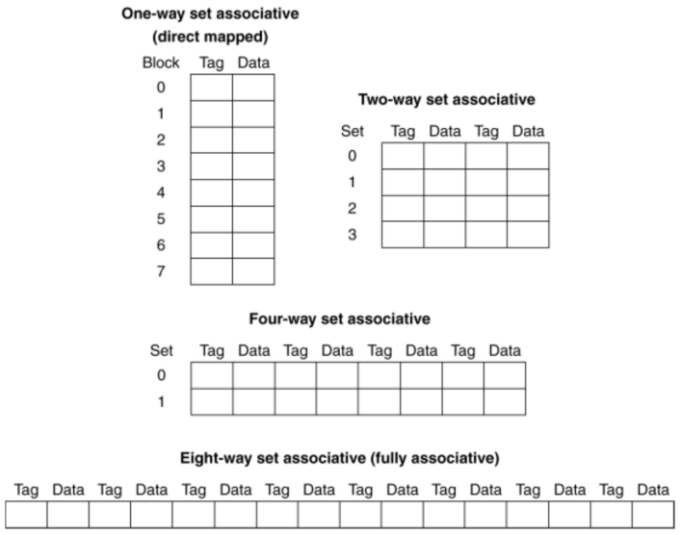


Figure 22.2 — Spectrum of associativity for an 8-entry cache (slide 40)

Replacement (slide 45): direct mapped has no choice. Set associative: prefer a **non-valid** entry; otherwise **LRU** (least-recently used — simple for 2-way, manageable for 4-way, too hard beyond) or **random** (about as good as LRU at high associativity).

How much associativity? (slide 43) 64 KB D-cache, 16-word blocks: 1-way 10.3 %, 2-way 8.6 %, 4-way 8.3 %, 8-way 8.1 % — **diminishing returns**. Higher associativity lowers miss rate but increases cost and access time.

2. Worked Example: Four-Block Caches, Sequence 0, 8, 0, 6, 8 (slides 41–42)

Direct mapped (index = block mod 4):

Block	Index	Result	0	1	2	3
0	0	miss	Mem[0]			
8	0	miss	**Mem[8]**			
0	0	miss	**Mem[0]**			
6	2	miss	Mem[0]		Mem[6]	
8	0	miss	**Mem[8]**		Mem[6]	

2-way set associative (set = block mod 2, LRU):

Block	Set	Result	Set 0	Set 0	Set 1
0	0	miss	Mem[0]		
8	0	miss	Mem[0]	Mem[8]	
0	0	**hit**	Mem[0]	Mem[8]	
6	0	miss	Mem[0]	**Mem[6]** (8 was LRU)	
8	0	miss	**Mem[8]** (0 was LRU)	Mem[6]	

Fully associative: 0 miss, 8 miss, 0 **hit**, 6 miss, 8 **hit**.

MEMORIZE **Misses: direct mapped 5, 2-way 4, fully associative 3.** More associativity removes **conflict** misses.

Exam Approach — LRU Tables

1. #sets = #blocks ÷ ways; **set = block address mod #sets**.
2. Write one row per access: block, set, hit/miss, **full content after the access**.
3. Keep an **LRU order** for each set (write it in your rough work, e.g. "set 0: 0 < 8" meaning 0 is LRU). A hit **refreshes** the block.
4. On a miss in a full set, replace the **LRU** block and say which one was replaced.
5. Finish with miss count and hit ratio for each organization.

TRAP

- With a fully associative cache **all** blocks compete in one set — never use mod.
- Byte addresses must first be converted to **block** addresses.
- A hit changes the LRU order even though the content does not change.

Practice — Lesson 22

(a) Actual — 2022-23 (CSE 305) Q2(a). (b) Actual — 2021-22 Q4(a). (c) Actual — 2019-20 Q6(a). (d) Actual — 2017-18 Q4(b). (e) Actual — 2013-14 Q1(b).

(a) Two caches of **eight blocks**: one direct mapped, one 2-way set associative (LRU). Build the table (address, index, hit/miss, content after access) for block addresses **32, 36, 35, 32, 48, 32, 51, 35**, and compute each hit ratio.

(b) Three caches of **four one-word blocks** (LRU): fully associative, 2-way, direct mapped. Find the number of misses for **1, 3, 2, 4, 5, 1, 3, 1, 3, 5, 3, 2**.

(c) Three caches of **eight one-word blocks**: direct mapped, 2-way, 4-way (LRU). Complete the table for **0, 12, 16, 18, 0**.

(d) Same caches as (b) for the sequence **1, 3, 5, 1, 3, 1, 3, 5, 3, 1, 5, 3**.

(e) A 2-way set-associative cache with **four blocks**, empty, LRU. How many hits does **0, 2, 4, 0, 2, 4, 0, 2, 4** give?

✓ Solution

(a) Direct mapped (index = block mod 8):

Block	Index	Result	Valid content after access
32	0	miss	[0] 32
36	4	miss	[0] 32, [4] 36
35	3	miss	[0] 32, [3] 35, [4] 36
32	0	**hit**	[0] 32, [3] 35, [4] 36
48	0	miss (replace 32)	[0] 48, [3] 35, [4] 36
32	0	miss (replace 48)	[0] 32, [3] 35, [4] 36
51	3	miss (replace 35)	[0] 32, [3] 51, [4] 36
35	3	miss (replace 51)	[0] 32, [3] 35, [4] 36

1 hit / 8 → hit ratio 12.5 %.

2-way (4 sets, set = block mod 4):

Block	Set	Result	Content after access (LRU first)
32	0	miss	S0: 32
36	0	miss	S0: 32, 36
35	3	miss	S0: 32, 36 · S3: 35
32	0	**hit**	S0: 36, 32 · S3: 35
48	0	miss (replace 36)	S0: 32, 48 · S3: 35
32	0	**hit**	S0: 48, 32 · S3: 35
51	3	miss	S0: 48, 32 · S3: 35, 51
35	3	**hit**	S0: 48, 32 · S3: 51, 35

3 hits / 8 → hit ratio 37.5 %.

(b)

Block	1	3	2	4	5	1	3	1	3	5	3	2	Misses
FA	M	M	M	M	M(1)	M(3)	M(2)	H	H	H	H	M(4)	**8**
2-way	M	M	M	M	M(1)	M(3)	M(5)	H	H	M(1)	H	H	**8**
DM	M	M	M	M	M(1)	M(5)	H	H	H	M(1)	H	H	**7**

M(x) = miss replacing block x. Final contents — FA: 1, 5, 3, 2 · 2-way: set 0 {4, 2}, set 1 {5, 3} · DM: [0] 4, [1] 5, [2] 2, [3] 3. Here the direct-mapped cache wins: the LRU caches are hurt by the cyclic pattern 1, 3, 5 / 1, 3, 2, 4, 5.

(c)

Cache	0	12	16	18	0	Misses
Direct mapped (index mod 8)	M [0]	M [4]	M [0] repl. 0	M [2]	M [0] repl. 16	**5**
2-way (set mod 4)	M S0	M S0	M S0 repl. 0	M S2	M S0 repl. 12	**5**
4-way (set mod 2)	M S0	M S0	M S0	M S0	**Hit**	**4**

Final: DM [0] 0, [2] 18, [4] 12 · 2-way S0 {16, 0}, S2 {18} · 4-way S0 {0, 12, 16, 18}.

(d) Fully associative: **3 misses** (only the first 1, 3, 5; the cache holds all three). 2-way: all odd blocks go to set 1 and three blocks fight for two places → **9 misses** (hits on the 6th, 7th and 9th accesses). Direct mapped: 3 is alone in index 3, 1 and 5 fight for index 1 → **7 misses**.

(e) 2 sets; 0, 2 and 4 are all even → **all in set 0**, which holds only two. With LRU every access evicts the block needed next → **0 hits** (9 misses). Final set 0: {2, 4}.

Lesson 23 — Writes, Block Size and Miss Types

Slides: Chapter 5, 28–32 and 77–79

EXAM FOCUS **2023-24 Q2(c):** advantages/disadvantages of larger blocks, early restart and critical-word-first, uses of the write buffer in write-through and write-back. **2017-18 Q3(b)** / 2015-16: write-through vs write-back and how each handles a **write miss**. **2017-18 Q3(a):** three sources of misses.

1. Write Policies (slides 30–32)

	Write-through	Write-back
On a write hit	Update the cache **and** memory	Update the cache only; mark the block **dirty**
Memory consistent?	Always	Only when a dirty block is written back on replacement
Cost	Every write goes to memory — e.g. base CPI 1, 10 % stores, 100-cycle write → CPI = 1 + 0.1 × 100 = **11**	Extra state (dirty bit); a replacement may need a write-back first
Write buffer	Holds data waiting to be written to memory; the CPU continues immediately and stalls only if the buffer is **full**	Holds the dirty block being evicted so the **new block can be read first**
Write miss	**Write allocate** (fetch the block) or **write around / no write allocate** (don't fetch — programs often write a whole block before reading it)	Usually **fetch the block** (write allocate)

Virtual memory uses only write-back — disk writes take millions of cycles (slides 58, 77).

2. Block Size Trade-off (slide 28)

- **Larger blocks reduce the miss rate** (spatial locality)…
- …but in a fixed-size cache there are **fewer blocks** → more competition → miss rate can rise; large blocks also bring unused data (**pollution**).
- **Larger miss penalty** (more bytes to transfer) can override the benefit.
- **Early restart:** resume the CPU as soon as the requested word arrives, not after the whole block. **Critical word first:** fetch the requested word first, then the rest of the block. Both reduce the effective miss penalty of large blocks.

3. Cache Misses and the 3 Cs (slides 29, 78–79)

On a miss: **stall the pipeline**, fetch the block from the next level; for an I-cache miss **restart the instruction fetch**, for a D-cache miss **complete the data access**.

Miss type	Cause	Reduced by (and its cost)
Compulsory (cold start)	First access to a block	Larger blocks (↑ miss penalty; pollution if very large)
Capacity	Cache too small; a replaced block is accessed again	Larger cache (may ↑ access time)
Conflict (collision)	Competition for entries in a set — would not occur in a fully associative cache of the same size	Higher associativity (may ↑ access time)

Exam Approach

- For "advantages and disadvantages" questions, give **both sides** with one line each and the fix (early restart / critical word first; write buffer).
- For a miss-classification question: first touch = compulsory; would also miss in a **fully associative** cache of the same size = capacity; otherwise = conflict.

Practice — Lesson 23

(a) Actual — 2023-24 Q2(c). (b) Actual — 2017-18 Q3(b). (c) Actual — 2012-13 Q6(d).

(a) What are the advantages and disadvantages of larger cache blocks? What is meant by early restart and critical-word-first? What are the uses of the write buffer in the write-through and write-back schemes?

(b) Briefly explain write-through and write-back. How do they handle a write miss?

(c) A program alternates between two blocks starting at **0x1000** and **0x2000**, accessing each 100 times. The system has a **1 KB direct-mapped cache with 32 blocks**. How many misses occur, and how many are compulsory and conflict misses?

✓ Solution

(a) Advantages: exploit spatial locality → lower miss rate; fewer tags per byte of data. **Disadvantages:** fewer blocks in a fixed-size cache → more conflicts, pollution; **larger miss penalty**. **Early restart:** the CPU continues as soon as the requested word arrives. **Critical word first:** memory sends the requested word first. **Write buffer:** in write-through it absorbs every write so the CPU need not wait for memory (stall only when full); in write-back it holds an evicted dirty block so the missing block can be read first and the old one written afterwards.

(b) See the table in §1: write-through updates cache and memory on every write (often with a write buffer) and on a write miss either allocates (fetches the block, then writes) or writes around (writes only memory). Write-back writes only the cache, keeps a dirty bit, writes the block to memory when it is replaced, and on a write miss normally fetches the block and then writes it in the cache.

(c) Block size = 1 KB / 32 = 32 bytes. Index = (address / 32) mod 32: 0x1000 / 32 = 128 → index **0**; 0x2000 / 32 = 256 → index **0**. Both blocks map to the same entry and evict each other on every access → **200 misses**: **2 compulsory** (first touch of each block) and **198 conflict** (a fully associative cache would hold both, so they are not capacity misses).

Lesson 24 — Cache Performance: CPI, AMAT and Multilevel Caches

Slides: Chapter 5, 33–37 and 46–49

EXAM FOCUS **Asked in nearly every paper.** Multilevel speedup: **2023-24 Q2(a)**, **2021-22 Q3(a)**, **2022-23 (CSE 305) Q3(a)**, 2017-18 Q4(a), 2015-16, 2012-13. Two processors with different hierarchies: **2022-23 (CSE 209) Q9(a)**. I-/D-cache CPI: 2017-18 Q3(d), 2015-16, 2014-15. Four-level probabilities: 2020-21 Q7.

1. Formulas (slide 33)

CPU time = (CPU execution cycles + Memory-stall cycles) × Clock cycle time

Memory-stall cycles = (Instructions / Program) × (Misses / Instruction) × Miss penalty

Effective CPI = Base CPI + Σ (misses per instruction × penalty in cycles)

AMAT = Hit time + Miss rate × Miss penalty

$$\text{CPU time} = (\text{CPU execution clock cycles} + \text{Memory-stall clock cycles}) \times \text{Clock cycle time}$$

$$\text{Memory-stall clock cycles} = (\text{Read-stall cycles} + \text{Write-stall cycles})$$

$$\text{Read-stall cycles} = \frac{\text{Reads}}{\text{Program}} \times \text{Read miss rate} \times \text{Read miss penalty}$$

$$\text{Write-stall cycles} = \left(\frac{\text{Writes}}{\text{Program}} \times \text{Write miss rate} \times \text{Write miss penalty}\right) + \text{Write buffer stalls}$$

$$\text{Memory-stall clock cycles} = \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

$$\text{Memory-stall clock cycles} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

Figure 24.1 — Measuring cache performance (slide 33)

2. Worked Example: I-cache and D-cache (slides 34–35)

I-cache miss rate 2 %, D-cache miss rate 4 %, miss penalty 100 cycles, base CPI 2, loads & stores 36 % of instructions.

I-cache stalls = 0.02 × 100 = 2 D-cache stalls = 0.36 × 0.04 × 100 = 1.44

Actual CPI = 2 + 2 + 1.44 = **5.44** → the ideal (perfect-cache) CPU is 5.44 / 2 = **2.72× faster**

If the base CPI improves to 1 (same memory): CPI = 1 + 3.44 = 4.44 → ideal is **4.44× faster**; the time spent on memory stalls rises from 3.44/5.44 = **63 %** to 3.44/4.44 = **77 %**. Faster CPUs make miss penalties **more significant** (slide 37).

MEMORIZE Every instruction makes **one instruction fetch**; only loads/stores make a **data** access — multiply the D-cache miss rate by the load/store fraction.

AMAT example (slide 36): 1 ns clock, hit time 1 cycle, miss penalty 20 cycles, miss rate 5 % → AMAT = 1 + 0.05 × 20 = **2 ns** (2 cycles).

3. Worked Example: Adding an L2 Cache (slides 46–48)

Base CPI 1, 4 GHz (0.25 ns cycle), miss rate per instruction 2 %, main memory 100 ns.

Miss penalty to memory = 100 / 0.25 = 400 cycles	CPI (L1 only) = 1 + 0.02 × 400 = **9**

Add L2: access 5 ns, global miss rate to memory 0.5 %.

L2 hit penalty = 5 / 0.25 = 20 cycles	
CPI = 1 + 0.02 × 20 + 0.005 × 400 = **3.4**	Performance ratio = 9 / 3.4 = **2.6**

MEMORIZE **Pattern:** CPI = base + (L1 miss rate × L2 time) + (global L2 miss rate × L3 time) + … + (global miss rate to memory × memory time). Each level's time applies to **every access that reaches it**. Convert ns to cycles with the clock period first.

Design focus (slide 49): L1 — minimal **hit time** (small, usually smaller blocks); L2 — low **miss rate** (larger, larger blocks; its hit time matters less).

TRAP

- **Global** miss rate (fraction of all accesses that reach memory) vs **local** miss rate (fraction of accesses *to that cache* that miss). Global = product of local rates down the chain.
- "Access time for either a hit or a miss" means every access reaching that level pays its time.
- Answer "how much faster" as a ratio of CPIs (same clock) — and say what it is compared with.

Practice — Lesson 24

(a) Actual — 2023-24 Q2(a). (b) Actual — 2021-22 Q3(a). (c) Actual — 2022-23 (CSE 305) Q3(a). (d) Actual — 2017-18 Q3(d). (e) Actual — 2022-23 (CSE 209) Q9(a). (f) Actual — 2020-21 Q7.

(a) Base CPI 1.0 (all references hit in L1), 4 GHz, main memory 200 ns including miss handling, L1 miss rate per instruction 5 %. (i) How much faster with an L2 of 10 ns (hit or miss) that reduces the miss rate to memory to 2 %? (ii) How much faster if we further add an L3 of 40 ns that reduces it to 1 %?

(b) Same as (a) but L1 miss rate 4 %, L2 → 1 %, L3 → 0.5 %.

(c) Base CPI 2.0, clock 0.5 ns, memory 150 ns, L1 miss rate 5 %. (i) Total CPI with one level; how much faster is the ideal CPU? (ii) Add L2 of 15 ns reducing the miss rate to 0.5 %: total CPI? (iii) Speedup of two levels over one?

(d) I-cache miss rate 3 %, D-cache 5 %, loads and stores 40 %, CPI 2 without stalls, miss penalty 200 cycles. How much faster is a perfect cache? What fraction of time is spent on memory stalls?

(e) P1 has L1 + L2, P2 has L1 + L2 + L3. DRAM 100 ns; memory accesses are 30 % of instructions; L2 hit time = 20 × cycle time for both; base CPI 1. L1: P1 4 % miss, P2 3 %. L2 (local miss rate, hit time): P1 65 %, 27.5 ns; P2 50 %, 30.5 ns. L3 (P2): global miss rate 1 %, hit time 50.7 ns. (i) Which processor is faster? (ii) What L2 miss rate would the slower one need to match the faster one?

(f) A hierarchy Cache → RAM → SSD → HDD has local miss rates 5 %, 15 %, 25 %, 0 % and access times 1, 10, 100, 1000 cycles. (a) Probability that data is fetched from the SSD? (b) Expected time of a fetch? (c) If the cache is split into I- and D-caches, overall miss rate stays 5 %, D-cache miss rate is 10 % and 10 % of instructions access data, what is the I-cache miss rate?

✓ Solution

(a) Cycle = 0.25 ns → memory = 200 / 0.25 = **800 cycles**; L2 = 10 ns = **40 cycles**; L3 = 40 ns = **160 cycles**.

$CPI_{L1} = 1 + 0.05 \times 800 =$ **41**
$CPI_{L1+L2} = 1 + 0.05 \times 40 + 0.02 \times 800 = 1 + 2 + 16 =$ **19** → 41 / 19 = **2.16× faster**
$CPI_{L1+L2+L3} = 1 + 0.05 \times 40 + 0.02 \times 160 + 0.01 \times 800 = 1 + 2 + 3.2 + 8 =$ **14.2** → 19 / 14.2 = **1.34×** over L1+L2 (41 / 14.2 = **2.89×** over L1 only)

(b)

$CPI_{L1} = 1 + 0.04 \times 800 = 33$ $CPI_{L1+L2} = 1 + 0.04 \times 40 + 0.01 \times 800 =$ **10.6** → **3.11×**
$CPI_{L1+L2+L3} = 1 + 1.6 + 0.01 \times 160 + 0.005 \times 800 =$ **8.2** → **1.29×** over L1+L2 (**4.02×** over L1 only)

(c) Memory = 150 / 0.5 = 300 cycles; L2 = 15 / 0.5 = 30 cycles.

(i) CPI = 2 + 0.05 × 300 = **17** → ideal CPU is 17 / 2 = **8.5× faster**
(ii) CPI = 2 + 0.05 × 30 + 0.005 × 300 = 2 + 1.5 + 1.5 = **5**
(iii) Speedup = 17 / 5 = **3.4**

(d) I-stalls = 0.03 × 200 = 6; D-stalls = 0.40 × 0.05 × 200 = 4.

CPI = 2 + 6 + 4 = **12** → perfect cache is 12 / 2 = **6× faster**; stall fraction = 10 / 12 = **83.3 %**

(e) *Assumptions (state them):* cycle time = L2 hit time / 20; L1 hits are covered by the base CPI; each instruction makes **1.3 memory accesses** (1 fetch + 0.3 data) and the miss rates are per access.

	P1	P2
Cycle time	27.5 / 20 = 1.375 ns	30.5 / 20 = 1.525 ns
Memory (cycles)	100 / 1.375 = 72.7	100 / 1.525 = 65.6
L3 hit (cycles)	—	50.7 / 1.525 = 33.2
Stall cycles per access	0.04 × (20 + 0.65 × 72.7) = **2.69**	0.03 × 20 + 0.03 × 0.5 × 33.2 + 0.01 × 65.6 = **1.75**
Effective CPI	1 + 1.3 × 2.69 = **4.50**	1 + 1.3 × 1.75 = **3.28**
Time per instruction	4.50 × 1.375 = **6.19 ns**	3.28 × 1.525 = **5.00 ns**

(i) P2 is faster (6.19 / 5.00 ≈ 1.24×). **(ii)** P1 must reach 5.00 ns → CPI = 5.00 / 1.375 = 3.64 → stalls per access = (3.64 − 1)/1.3 = 2.03 = 0.04 × (20 + m × 72.7) → **m ≈ 42 %** L2 local miss rate (down from 65 %).

> **NOTE** If you instead treat the 30 % as the only accesses (0.3 per instruction), P1 = 2.49 ns, P2 = 2.33 ns — **P2 is still faster** and P1 would need an L2 miss rate of about 52 %. Write your assumption first; the method is what earns the marks.

(f)

(a) Reaches SSD = 0.05 × 0.15 = 0.0075; served by SSD = 0.0075 × 0.75 = **0.005625** (0.56 %)
(b) Expected time = 1 + 0.05 × (10 + 0.15 × (100 + 0.25 × 1000)) = 1 + 0.05 × 62.5 = **4.125 cycles**

(c) Accesses per instruction = 1 + 0.1 = 1.1; $0.05 \times 1.1 = m_I \times 1 + 0.10 \times 0.1 \rightarrow m_I =$ **4.5 %**

Lesson 25 — Software and the Cache

Slides: Chapter 5, 50–55 and 90–91

EXAM FOCUS **2022-23 (CSE 209) Q12(d):** explain why the clock-cycle graph of radix sort vs quicksort does not follow the instruction-count graph (slide 51). Pitfalls (byte vs word addressing) are useful short answers.

1. Misses Depend on the Access Pattern (slide 51)

For large arrays **radix sort executes fewer instructions per item** than quicksort, but its **clock cycles per item** rise again: radix sort's access pattern has poor locality, so its **cache misses per item** grow sharply once the data no longer fits in the cache, while quicksort keeps working on small contiguous sub-arrays and stays cache-friendly.

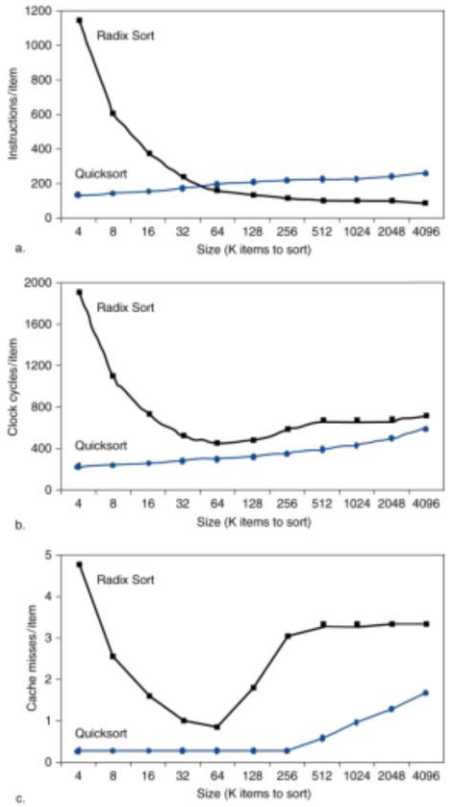


Figure 25.1 — Radix sort vs quicksort: instructions, clock cycles and cache misses per item (slide 51)

EXAM-READY ANSWER Figure 12.2 does not follow Figure 12.1 because clock cycles depend not only on the instruction count but also on **memory-stall cycles**. For large inputs radix sort suffers many more **cache misses per item** (poor spatial/temporal locality), so the time saved by executing fewer instructions is lost to miss penalties. Quicksort's misses stay low, so its clock cycles track its instruction count.

2. Blocking (slides 52–55)

Goal: maximize accesses to data **before it is replaced**. In matrix multiply (DGEMM) the inner loop sweeps whole rows/columns; for large n they do not fit in the cache and are evicted before reuse. **Blocking** works on BLOCKSIZE × BLOCKSIZE sub-matrices that fit in the cache, so each loaded element is reused many times (temporal locality). On the slide's measurements blocked DGEMM keeps its performance as the matrices grow, while the unoptimized version drops.

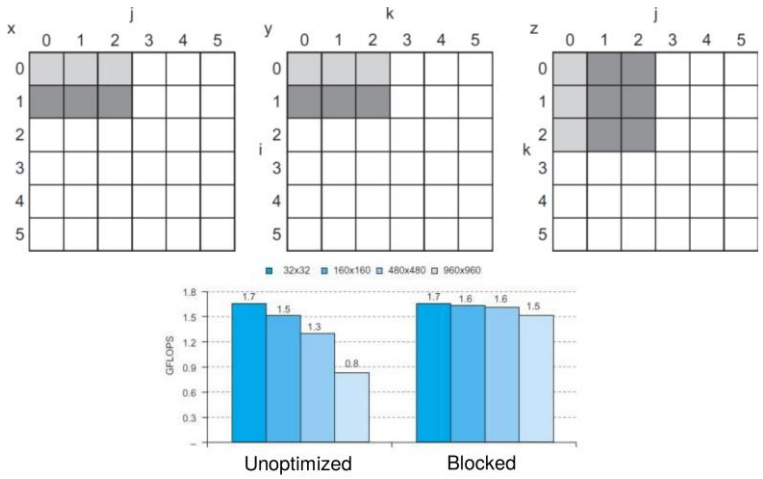


Figure 25.2 — Unoptimized vs blocked DGEMM access patterns (slides 53, 55)

3. Out-of-Order CPUs and Pitfalls (slides 50, 90–91)

- Out-of-order CPUs keep executing **independent** instructions during a miss → the effect of a miss depends on data flow; evaluate by **simulation**, not by AMAT alone.
- **Byte vs word addressing:** 32-byte direct-mapped cache, 4-byte blocks → **byte 36** maps to block (36/4) mod 8 = **1**, **word 36** maps to block 36 mod 8 = **4**.
- **Large strides** (e.g. iterating over columns of a row-major array) give poor spatial locality.
- Shared L2/L3 with **less associativity than cores** → conflict misses; more cores need more associativity.

Lesson 26 — Virtual Memory and Page Tables

Slides: Chapter 5, 56–62 · multilevel and inverted page tables follow the textbook

EXAM FOCUS **Page-table size** is asked repeatedly: **2022-23 (CSE 209) Q10(a)** (single, three-level, inverted), 2022-23 (CSE 305) Q3(c), 2018-19 Q3(b), 2013-14, 2014-15. Page-table diagram: 2021-22 Q3(b), 2017-18 Q3(c). Page replacement with reference bits: **2020-21 Q8**. Purpose of virtual addresses: 2022-23 (CSE 305) Q3(b).

1. The Idea (slide 56)

Use main memory as a **"cache" for disk**, managed jointly by CPU hardware and the OS.

- Programs share main memory; each gets a **private virtual address space** → **protection** and **relocation**; programs can be larger than physical memory.
- The CPU and OS translate **virtual addresses** → **physical addresses**.
- A VM "block" is a **page**; a VM "miss" is a **page fault**.

Cache term	Virtual-memory term
Block	Page
Miss	Page fault
Tag	Virtual page number (VPN)
Block placement	Fully associative (any physical page)

2. Address Translation (slides 57, 60)

```
Virtual address :  [ Virtual page number  |  Page offset ]
                              | (translate)        | (copy unchanged)
Physical address:  [ Physical page number |  Page offset ]
```

- Page offset bits = $\log_2$ (page size) — **not translated**.
- **Page table:** an array of page-table entries (PTEs) **indexed by VPN**; the **page table register** points to its start in physical memory.
- PTE (page present) = **valid bit + physical page number** + status bits (reference/use, dirty, protection). If not present, the PTE can hold the page's location in **swap space on disk**.

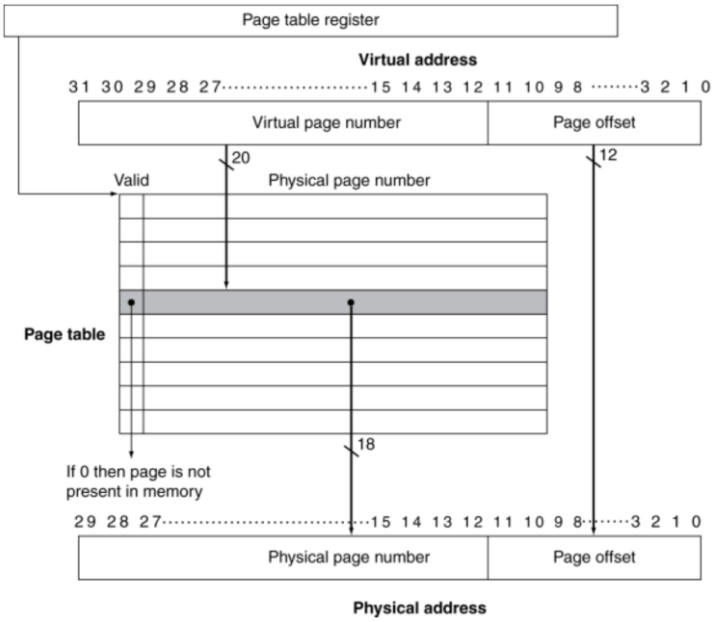


Figure 26.1 — Translation using a page table (4 KB pages, 32-bit VA, 30-bit PA) (slide 60)

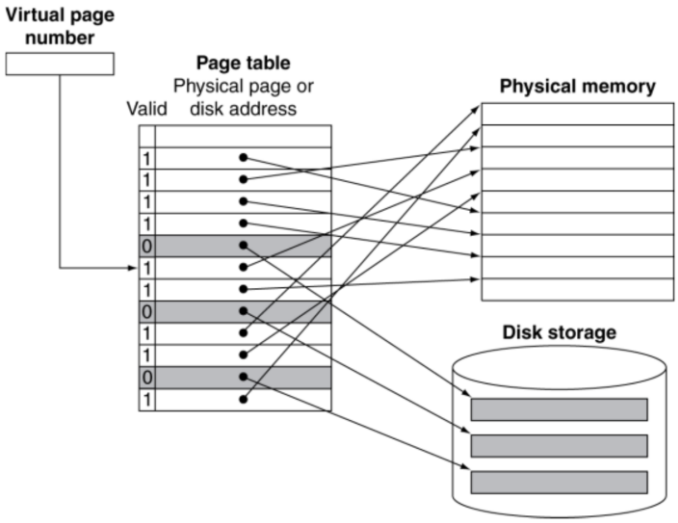


Figure 26.2 — Mapping pages to physical memory or disk (slide 61)

3. Page Faults, Replacement and Writes (slides 58, 62, 67)

- A page fault costs **millions of cycles** (fetch from disk, handled by OS code) → minimize the fault rate: **fully associative placement**, smart replacement, and **write-back** (never write-through).
- **LRU approximation:** a **reference (use) bit** in each PTE is set on access and periodically cleared by the OS; a page with reference bit 0 has not been used recently → good victim.
- **Dirty bit:** set when the page is written; only dirty pages are written back to disk.
- **Page fault handler:** use the faulting VA to find the PTE → locate the page on disk → choose a page to replace (if dirty, write it to disk first) → read the page in and update the page table → make the process runnable → **restart the faulting instruction**.

4. Page-Table Size

Entries = 2^(VA bits − offset bits)	**Size = entries × PTE size**

Example (2013-14): 32-bit VA, 8 KB pages, 4-byte PTE → offset 13 → 2^{19} = **524,288 entries** → **2 MiB** per process.

Multilevel page table (textbook)

Split the VPN into several fields; the first indexes a top-level table whose entries point to second-level tables, and so on. Only tables for **regions actually in use** are allocated, so memory is small for sparse address spaces (best case: one table per level), but a full address space needs slightly **more** than a single-level table.

Inverted page table (textbook)

One entry per physical page (not per virtual page). Each entry stores which ⟨process ID, VPN⟩ occupies that frame; lookup uses hashing. Size depends only on **physical** memory and is shared by all processes.

TRAP

- Physical memory size does not change a forward page table's size — only VA bits, page size and PTE size do.
- Multiply by the **number of processes** when every process has its own table.
- Use **binary** units: 16 KB = 2^{14} B, 8 GB = 2^{33} B.

Practice — Lesson 26

(a) Actual — 2022-23 (CSE 209) Q10(a). (b) Actual — 2018-19 Q3(b)(a–b). (c) Actual — 2022-23 (CSE 305) Q3(b)–(c). (d) Actual — 2020-21 Q8.

(a) Virtual address 38 bits, physical DRAM 8 GB, page size 16 KB, PTE 4 bytes, **75 processes**, PID 8 bits. (i) Memory for single-level page tables? (ii) Three-level page tables with equal bits per level — best, average and worst case? (iii) Inverted page table, if each entry stores 4 bytes of extra information besides the required fields?

(b) 40-bit virtual byte address, 16 KB pages, 36-bit physical byte address; valid, protection, dirty and use bits; disk addresses not stored. (a) Page-table size? (b) Size with an inverted page table?

(c) What is the difference between a virtual and a physical address, and why use virtual addresses? For a 64-bit VA mapped to a 30-bit PA with 8 KB pages, how many page-table entries are there?

(d) Approximate LRU with one reference bit per page-table entry. Virtual memory has 8 pages, the (fully associative) page table has 4 entries, and reference bits are cleared after every four time steps. Simulate the references **0, 0, 2, 7, 0, 6, 1, 2, 6, 7, 1, 1, 7, 1, 4, 1**, marking hits and misses.

✓ Solution

(a) Offset = 14 bits → VPN = 38 − 14 = **24 bits**.

> (i) Per process = 2^{24} × 4 B = 64 MiB; × 75 = 4800 MiB = **4.69 GiB**

(ii) 24 / 3 = **8 bits per level** → every table has 256 entries × 4 B = **1 KiB**.

Case	Tables per process	× 75 processes
Best (one entry used per level)	1 + 1 + 1 = 3 tables = **3 KiB**	**225 KiB**
Average (assume half of each table used)	1 + 128 + 128 × 128 = 16,513 tables ≈ **16.1 MiB**	≈ **1.18 GiB**
Worst (whole space used)	1 + 256 + 65,536 = 65,793 tables ≈ **64.25 MiB**	≈ **4.71 GiB**

(iii) Entries = physical pages = 8 GB / 16 KB = 2^{19} = **524,288** (one table for all processes). Required fields = PID 8 + VPN 24 = 32 bits = 4 B; plus 4 B extra = **8 B per entry**.

> Inverted table = 2^{19} × 8 B = **4 MiB**

(b) Offset 14 → VPN = 26 bits → 2^{26} entries. PPN = 36 − 14 = 22 bits; PTE = 22 + 4 status bits = 26 bits.

> (a) 2^{26} × 26 bits = 1,744,830,464 bits = **208 MiB** (256 MiB if each PTE is rounded to 4 bytes)
>
> (b) Inverted: 2^{22} physical pages × (VPN 26 + 4 status bits = 30 bits) = **15 MiB**

(c) A **virtual address** is generated by the program in its own private address space; a **physical address** is the actual location in main memory. Virtual addressing lets many programs share memory safely (protection), lets each program be loaded anywhere (relocation) and allows programs larger than physical memory. Entries = $2^{(64\ -\ 13)}$ = $\mathbf{2^{51}}$ (one per virtual page).

(d) Frames in table order; replacement takes the **first entry whose reference bit is 0** (state this assumption). Bits are cleared before steps 5, 9 and 13.

Step	Page	Result	Frames after	Ref bits
1	0	miss	0 – – –	1 0 0 0
2	0	**hit**	0 – – –	1 0 0 0
3	2	miss	0 2 – –	1 1 0 0

Step	Page	Result	Frames after	Ref bits
4	7	miss	0 2 7 –	1 1 1 0
5	0	**hit** (bits cleared first)	0 2 7 –	1 0 0 0
6	6	miss	0 2 7 6	1 0 0 1
7	1	miss — replace 2	0 1 7 6	1 1 0 1
8	2	miss — replace 7	0 1 2 6	1 1 1 1
9	6	**hit** (cleared first)	0 1 2 6	0 0 0 1
10	7	miss — replace 0	7 1 2 6	1 0 0 1
11	1	**hit**	7 1 2 6	1 1 0 1
12	1	**hit**	7 1 2 6	1 1 0 1
13	7	**hit** (cleared first)	7 1 2 6	1 0 0 0
14	1	**hit**	7 1 2 6	1 1 0 0
15	4	miss — replace 2	7 1 4 6	1 1 1 0
16	1	**hit**	7 1 4 6	1 1 1 0

8 hits, 8 misses.

Lesson 27 — TLBs and the TLB–Page Table–Cache Path

Slides: Chapter 5, 63–71

EXAM FOCUS **2023-24 Q3(a)–(b):** TLB vs data cache + flowchart; **TLB/page-table trace**. **2021-22 Q3(c):** which TLB/page-table/cache combinations are possible. **2022-23 (CSE 209) Q10(b)–(c):** TLB + cache block diagram, context switch. **2019-20 Q7(b)–(c)**, **2017-18 Q4(c)–(d)**, 2018-19 Q3(a),(c), 2013-14: purpose of TLB, TLB miss handling, flowchart.

1. Why a TLB? (slide 63)

Translation would need **two memory accesses** per reference: one for the PTE, then the actual access. Page-table accesses have good locality, so keep a small **cache of PTEs** in the CPU: the **Translation Look-aside Buffer (TLB)**. Typical: 16–512 entries, 0.5–1 cycle hit, 10–100 cycles miss, 0.01 %–1 % miss rate.

- A TLB entry: **tag (VPN, or part of it)**, valid, dirty, reference bits, **physical page number**.
- The **tag** identifies *which* virtual page the entry translates — needed because the TLB holds only a few of the pages (2019-20 Q7(b)).
- "The TLB acts as a cache of the page table **for the entries that map to physical pages only**" — true: a page that is on disk is never in the TLB.

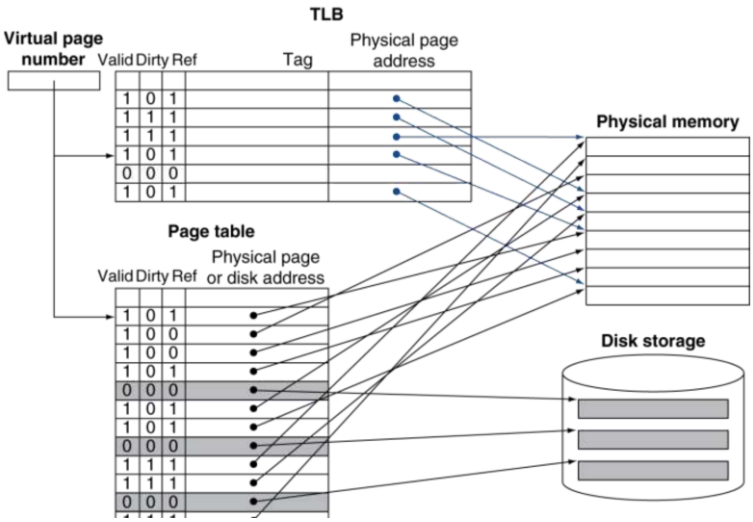


Figure 27.1 — Fast translation using a TLB (slide 64)

2. TLB Miss vs Page Fault (slides 65–67)

Event	Handling
TLB miss, page in memory	Load the PTE from the page table into the TLB (by hardware, or by software via a special exception with an optimized handler) and **retry** the access
TLB miss, page not in memory = page fault	OS fetches the page from disk, updates the page table (and TLB), then **restarts the faulting instruction**

3. TLB and Cache Together (slides 68–70)

A **physically indexed, physically tagged** cache: VA → TLB → PA → cache (index, tag compare). Data can be in the cache only if its page is in main memory; when the OS moves a page to disk it flushes that page from the cache and updates the page table and TLB.

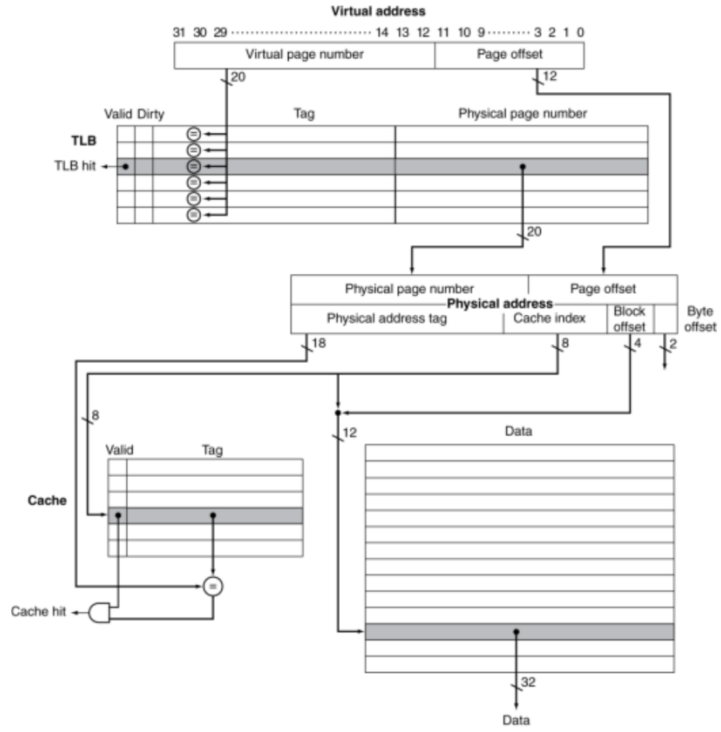


Figure 27.2 — TLB and cache interaction (slide 69)

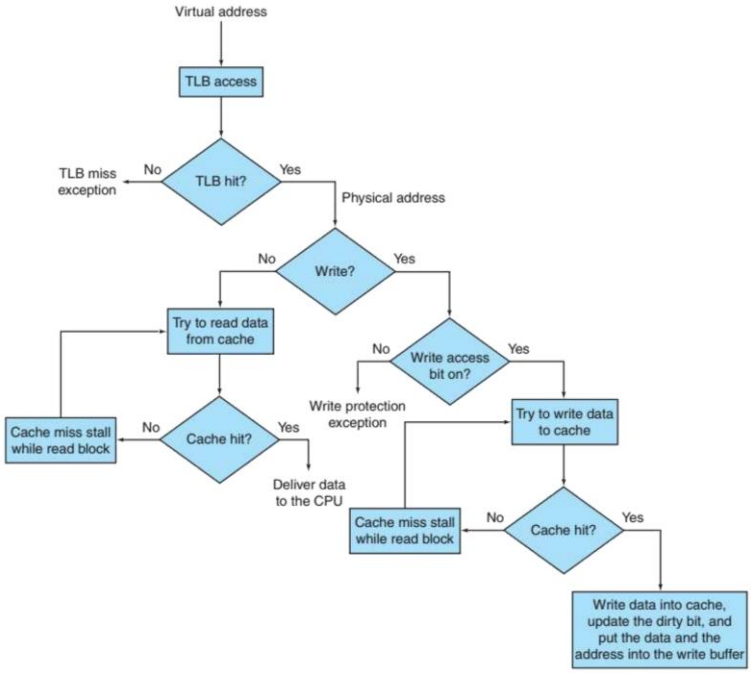


Figure 27.3 — Processing a read or write through the TLB and cache (slide 70)

Flowchart in words: TLB access → **miss** → TLB-miss exception. **Hit** → physical address. Read? → try the cache → hit: deliver data; miss: stall while the block is read. Write? → check the **write-access bit** (if off → write-protection exception) → try to write the cache → hit: write data into the cache, update the dirty bit and put data and address into the write buffer; miss: stall while the block is read, then write.

TLB vs data cache (2023-24 Q3(a))

	TLB	Data cache
Holds	Page-table entries (VPN → PPN + status bits)	Copies of memory blocks (data)
Indexed/tagged by	**Virtual** page number	**Physical** address (here)
Size	Tiny (16–512 entries), usually highly or fully associative	Much larger (KB–MB)
On a miss	Get the PTE from the page table (or page fault)	Get the block from the next memory level

Which combinations are possible? (slide 71, 2021-22 Q3(c))

TLB	Page table	Cache	Possible?
Hit	Hit	Miss	Possible (page table never checked on a TLB hit)
Miss	Hit	Hit	TLB miss, entry found in page table; after retry data is in the cache
Miss	Hit	Miss	TLB miss, entry found in page table; after retry data misses in the cache
Miss	Miss	Miss	TLB miss followed by a **page fault**; after retry data must miss in the cache
Hit	Miss	Miss	**Impossible:** no translation in the TLB if the page is not in memory
Hit	Miss	Hit	**Impossible:** same reason
Miss	Miss	Hit	**Impossible:** data cannot be in the cache if the page is not in memory

(Hit/Hit/Hit is the normal case.)

4. Protection and Context Switches (slide 72)

Hardware support for OS protection: **supervisor (kernel) mode**, privileged instructions, and page tables/state accessible only in supervisor mode. On a **context switch** the new process uses the same virtual addresses for different data, so TLB entries (and a **virtually** addressed cache) of the old process would give wrong translations → they must be flushed. To avoid flushing, tag each TLB entry (and virtual cache line) with a **process ID / address-space ID**, and use a **physically tagged** cache.

Exam Approach — TLB / Page-Table Trace

1. VPN = ⌊VA / page size⌋, offset = VA mod page size.
2. TLB hit? → done (update LRU). Else page table: valid → **TLB miss, page-table hit**; invalid → **page fault**, assign the next free physical page, mark the PTE valid.
3. On every TLB miss, insert the entry: use an **invalid** TLB slot first, else replace the **LRU** entry.
4. Physical address = PPN × page size + offset (if asked).

Practice — Lesson 27

(a) Actual — 2023-24 Q3(b) (same data as a textbook exercise). (b) Actual — 2022-23 (CSE 209) Q10(b). (c) Actual — 2019-20 Q7(c) / 2013-14. (d) Actual — 2018-19 Q3(c).

(a) 4 KB pages, 4-entry **fully associative** TLB, LRU. New pages from disk get the next physical page numbers 13, 14, 15, … VPN = ⌊VA / 4096⌋. Address stream: **4669, 2227, 13916, 34587, 48870, 12608, 49225**.

TLB Valid	Tag	PPN
1	11	12
1	7	4
1	3	6
0	4	9

Page table (VPN 0–11): 0 → 5 · 1 → disk · 2 → disk · 3 → 6 · 4 → 9 · 5 → 11 · 6 → disk · 7 → 4 · 8 → disk · 9 → disk · 10 → 3 · 11 → 12 (valid = 1 where a number is given).

(i) For each reference: TLB hit, page-table hit or page fault? (ii) Final TLB and page table.

(b) Draw the TLB–data cache block diagram with labels: page size 32 KB, 64-bit virtual address, 64 GB physical DRAM; TLB 32 entries, fully associative, with tag, valid, dirty, 24 bookkeeping bits and data bits; data cache 64 KB, direct mapped, 64 words per block, physically indexed and tagged.

(c) Explain how a TLB miss is handled.

(d) A processor has a 16-entry TLB and 4 KB pages. Programs access at least 2 MB of memory at a time. How will the system perform? How can it be improved with no extra money, and with more money?

✓ Solution

(a) *Assumption:* the initial LRU order is the listed order (top entry least recently used). The invalid entry is filled first.

Address	VPN	Offset	Result	PPN	TLB after (tag→PPN)
4669	1	573	TLB miss, **page fault** → new page 13	13	11→12, 7→4, 3→6, **1→13**
2227	0	2227	TLB miss, **page-table hit**	5	**0→5**, 7→4, 3→6, 1→13 (11 was LRU)
13916	3	1628	**TLB hit**	6	unchanged
34587	8	1819	TLB miss, **page fault** → page 14	14	0→5, **8→14**, 3→6, 1→13 (7 was LRU)
48870	11	3814	TLB miss, **page-table hit**	12	0→5, 8→14, 3→6, **11→12** (1 was LRU)
12608	3	320	**TLB hit**	6	unchanged
49225	12	73	TLB miss, **page fault** → page 15	15	**12→15**, 8→14, 3→6, 11→12 (0 was LRU)

Final TLB (all valid): 12 → 15, 8 → 14, 3 → 6, 11 → 12. **Final page table:** as given, plus **1 → 13, 8 → 14, 12 → 15** (now valid). Totals: 2 TLB hits, 2 page-table hits, 3 page faults.

(b) Field sizes:

Item	Bits
Page offset	32 KB → **15**
Virtual page number	64 − 15 = **49**
Physical address	64 GB → **36**; PPN = 36 − 15 = **21**
TLB entry (fully associative: tag = whole VPN)	tag 49 + valid 1 + dirty 1 + bookkeeping 24 + PPN 21 = **96 bits**; 32 entries = 3072 bits
Cache block	64 words = 256 B → offset **8** (6 word + 2 byte)
Cache index	64 KB / 256 B = 256 blocks → **8**
Cache tag	36 − 8 − 8 = **20**

```
Virtual address (64 bits):   [ VPN 49 | page offset 15 ]
  VPN ──→ TLB: 32 entries, fully associative, 32 comparators
          entry = V | D | 24 bookkeeping | tag 49 | PPN 21
      ──→ TLB hit + PPN (21 bits)
Physical address (36 bits):  [ PPN 21 | page offset 15 ]
                           = [ cache tag 20 | index 8 | block offset 8 ]
  index ──→ selects 1 of 256 cache lines (V | tag 20 | data 2048 bits)
  stored tag = address tag  AND  V ──→ Cache hit
  block offset ──→ mux selects the requested word
```

(c) A TLB miss means either the page is present but its PTE is not in the TLB, or the page is not present. The handler (hardware, or a software exception with an optimized handler in MIPS) uses the VPN to read the PTE from the page table (page-table register + VPN). If the PTE is valid, it copies the PTE into the TLB and **restarts the instruction**, which now hits. If the PTE is invalid, a **page fault** exception is raised and the OS loads the page from disk, updates the page table, and restarts the faulting instruction.

(d) TLB reach = 16 × 4 KB = **64 KB**, far below the 2 MB working set → the TLB **thrashes**: most accesses miss and need page-table walks, so performance is poor. **No money:** use **larger pages** (2 MB / 16 = 128 KB pages, or OS support for large/huge pages), and improve program locality. **More money:** a **larger TLB** (≥ 512 entries for 4 KB pages) or a **two-level TLB**, possibly with hardware page-table walking.

Lesson 28 — Cache Coherence

Slides: Chapter 5, 80–86

EXAM FOCUS **2023-24 Q3(c)** and **2021-22 Q3(d):** what is the cache coherence problem, and how does a snooping protocol solve it — with an example. 2013-14: three conditions for coherence. 2015-16: methods to maintain coherence.

1. The Problem (slide 80)

Two cores share a physical address space and each has a (write-through) cache:

Step	Event	CPU A's cache	CPU B's cache	Memory
0				0
1	CPU A reads X	0		0
2	CPU B reads X	0	0	0
3	CPU A writes 1 to X	1	**0 (stale)**	1

CPU B now reads the **old value 0** — the caches are **incoherent**.

2. Coherence Defined (slide 81)

Informally: **reads return the most recently written value.** Formally:

1. P writes X, then P reads X (no intervening writes) → the read returns the written value.
2. P_1 writes X, then P_2 reads X **sufficiently later** → the read returns the written value.
3. P_1 writes X and P_2 writes X → **all processors see the writes in the same order** (write serialization), ending with the same final value.

3. Protocols (slides 82–84)

Caches provide **migration** (data moves to the local cache → less shared-memory bandwidth) and **replication** of read-shared data (less contention).

	Snooping	Directory-based
Idea	Every cache **monitors (snoops) the bus** to see if others read or write blocks it holds	A **directory** records which caches hold a copy of each block
On a write by another core	Update or **invalidate** its own copy	Directory sends invalidations/updates only to the sharers
Best for	Small systems with a shared bus	Large systems — avoids flooding the bus with broadcasts

4. Worked Example: Invalidating Snooping Protocol (slide 85)

A cache gets **exclusive access** to a block when it is about to write it and **broadcasts an invalidate** on the bus; a later read by another cache **misses** and the **owning cache supplies** the updated value.

CPU activity	Bus activity	CPU A's cache	CPU B's cache	Memory
				0
CPU A reads X	Cache miss for X	0		0
CPU B reads X	Cache miss for X	0	0	0
CPU A writes 1 to X	**Invalidate for X**	1	(invalid)	0
CPU B reads X	Cache miss for X	1	1	1

(Write-back caches here: memory is updated when A supplies the block.)

EXAM-READY ANSWER The cache coherence problem arises when several processors cache the same memory block and one of them writes it, leaving stale copies in the other caches. In a snooping protocol every cache controller watches all bus transactions. Before writing, a processor broadcasts an invalidate for the block; every other cache holding that block invalidates its copy. When another processor later reads the block, it misses, and the cache holding the modified copy supplies the up-to-date value (memory is updated). Thus all reads return the most recently written value.

5. Memory Consistency (slide 86)

When are writes seen by other processors? Assume (1) a write completes only when all processors have seen it, and (2) a processor does not reorder writes with other accesses. Consequence: if P writes X then Y, any processor that sees the new Y also sees the new X. Processors may reorder reads, but not writes.

Quick Notes — All Lessons

Every lesson's revision points in one place. Figure numbers point back to the lessons.

L19 · Locality and Technology

- **Temporal:** recently used → used again soon (loops). **Spatial:** nearby → used soon (arrays, sequential code). Cache keeps recent blocks (temporal) and fetches whole blocks (spatial).
- Hit ratio = hits / accesses; miss ratio = 1 − hit ratio; miss penalty = time to bring the block.
- SRAM 0.5–2.5 ns · DRAM 50–70 ns (refresh, rows, burst, DDR, banks) · flash (nonvolatile, wear levelling; NOR random access, NAND block access, denser) · disk 5–20 ms.

$$\text{Disk read} = \text{seek} + 0.5 / (\text{rpm}/60) + \text{size} / \text{rate} + \text{controller}$$

- Slide: 4 + 2 + 0.005 + 0.2 = **6.2 ms**. 2017-18: 5 + 3 + 10 + 0.5 = **18.5 ms** → 15.5 ms with 2 ms seek (1.19×).

L20 · Direct-Mapped Caches

$$\text{Block address} = \lfloor \text{byte address} / \text{block size} \rfloor \quad \text{Index} = \text{block address} \bmod \#\text{blocks} \quad \text{Tag} = \text{remaining high bits}$$

- Valid bit 0 at power-on. Hit = valid **and** tag match. A miss into a valid entry = replacement.
- Slide trace 22, 26, 22, 26, 16, 3, 16, 18 → hits on the 2nd 22, 2nd 26, 2nd 16; 18 replaces 26.
- **2023-24 Q2(b):** 8-word blocks, 32 entries; 4 hits / 12 (33.3 %), 4 replacements; final [0] tag 3, [4] tag 2, [5] tag 0, [7] tag 0.

L21 · Fields and Total Bits

$$\text{Offset} = \log_2(\text{block bytes}) \quad \text{Index} = \log_2(\#\text{sets}) \quad \text{Tag} = \text{address bits} - \text{index} - \text{offset}$$

$$\text{Total bits} = \#\text{blocks} \times (\text{data} + \text{tag} + \text{valid} + \text{other bits})$$

- Doubling associativity: index −1, tag +1. Fully associative: no index.
- 16 KiB DM, 4-word blocks: tag 18, 147 bits/line → **147 Kibits**.
- 64 KB DM, 8-word: tag 16 → **559,104 bits**. 4K blocks, 4 words, 8-way: tag 19 → tag bits **77,824**, total **606,208**.
- 48-bit, 64 KB, 16-word, 4-way + V/D/P: tag 34, 549 bits/line → **562,176 bits**.
- 256 KB, 64-word, WT: overhead DM 15,360 · 2-way 16,384 · FA 25,600 bits.

L22 · Associativity and LRU

- DM: block mod #blocks · n-way: block mod #sets, n comparators · FA: anywhere, comparator per entry.
- Replacement: invalid entry first, then **LRU** (or random). Diminishing returns: 10.3 → 8.6 → 8.3 → 8.1 % (1/2/4/8-way).
- Slide 0, 8, 0, 6, 8 (4 blocks): DM 5 misses, 2-way 4, FA 3.
- 2022-23 (305) 32, 36, 35, 32, 48, 32, 51, 35: DM 1 hit (12.5 %), 2-way 3 hits (37.5 %).
- 2021-22 1, 3, 2, 4, 5, 1, 3, 1, 3, 5, 3, 2: FA 8, 2-way 8, DM 7 misses.
- 2019-20 0, 12, 16, 18, 0 (8 blocks): DM 5, 2-way 5, 4-way 4 misses.
- 2017-18 1, 3, 5, 1, 3, 1, 3, 5, 3, 1, 5, 3: FA 3, 2-way 9, DM 7 misses.

L23 · Writes and Miss Types

- **Write-through:** cache + memory; write buffer so the CPU stalls only when it is full; write miss → allocate or write around.
- **Write-back:** cache only, **dirty bit**, write on replacement; write buffer lets the new block be read first; write miss → usually allocate. VM always write-back.
- Larger blocks: ↓ miss rate (spatial) but fewer blocks, pollution, ↑ miss penalty → **early restart**, **critical word first**.
- **3 Cs:** compulsory (first access) · capacity (cache too small) · conflict (set competition; not in FA of same size).

L24 · Performance

CPI = base + Σ (misses/instruction × penalty) AMAT = hit time + miss rate × miss penalty

Multilevel: CPI = base + $m_{L1} \times t_{L2} + m_{global,L2} \times t_{L3/mem} + \ldots$

- Slide: I 2 %, D 4 %, 36 % ld/st, 100 cycles, base 2 → CPI 5.44 (2.72×). Base 1 → 4.44; stall share 63 % → 77 %.
- Slide L2: 4 GHz, 2 %, 100 ns → CPI 9; L2 5 ns, 0.5 % → 3.4 (2.6×).
- **2023-24:** 41 → 19 (2.16×) → 14.2 (1.34× more). **2021-22:** 33 → 10.6 (3.11×) → 8.2 (1.29×). **2022-23 (305):** 17 (ideal 8.5×) → 5 (3.4×). **2017-18 Q3(d):** CPI 12, 6× faster, 83.3 % stalls.
- **2022-23 (209) Q9(a):** P2 faster (5.00 vs 6.19 ns/instr with 1.3 accesses/instr); P1 needs L2 miss ≈ 42 %.
- Global vs local miss rate; convert ns → cycles first; L1 → fast hit time, L2 → low miss rate.

L25 · Software

- Radix vs quicksort: fewer instructions ≠ fewer cycles — radix sort's **cache misses** grow for large inputs.
- **Blocking** reuses sub-matrices before they are evicted.
- Byte 36 → block 1, word 36 → block 4 (32-byte cache, 4-byte blocks). Large strides → poor locality.

L26 · Virtual Memory

- Page = block, page fault = miss, fully associative placement, LRU via **reference bit**, write-back with **dirty bit**.
- VA = [VPN | offset] → PA = [PPN | offset]; page table indexed by VPN, located by the **page table register**; PTE = valid + PPN + status (or disk location).

Entries = 2^(VA bits − offset) Size = entries × PTE size (× processes)

- 32-bit, 8 KB, 4 B → 2^{19} entries, 2 MiB. 38-bit, 16 KB, 4 B, 75 processes → 4.69 GiB single-level; 3-level (8 bits/level, 1 KiB tables): best 225 KiB, worst ≈ 4.71 GiB; inverted ($2^{19} \times 8$ B) = **4 MiB**.
- Page-fault handler: find PTE → locate on disk → choose victim (write if dirty) → read page, update table → restart the instruction.

L27 · TLB

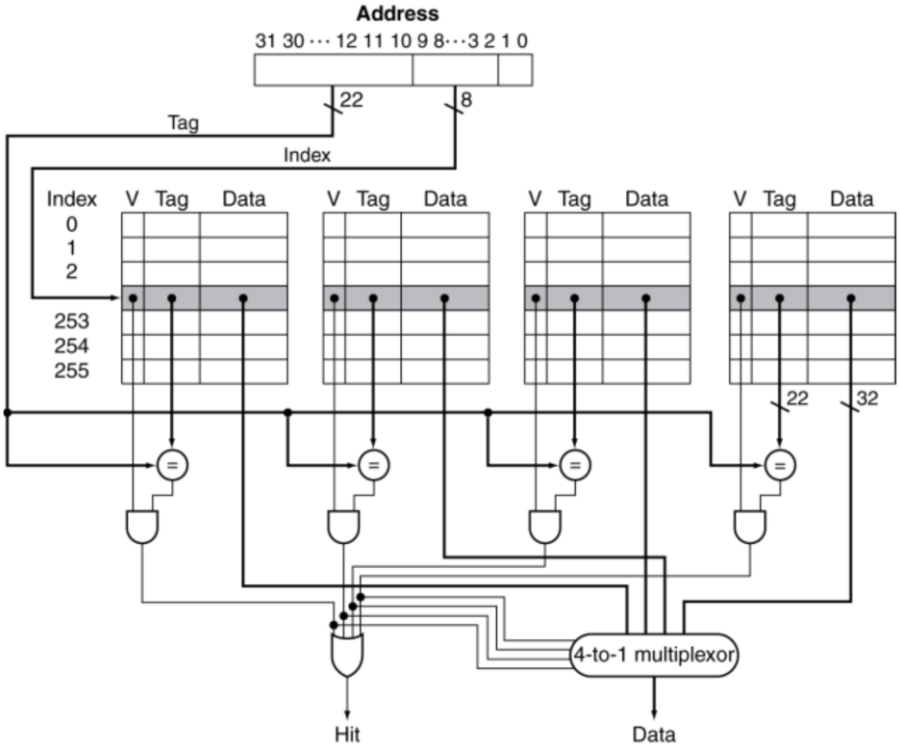
- TLB = cache of PTEs (16–512 entries); tag = VPN; only for pages in memory.
- TLB miss + page present → copy PTE, retry. Page not present → page fault → OS → restart.
- Impossible: **TLB hit with page-table miss**; **cache hit with page-table miss**.
- Context switch: flush TLB/virtual caches, or tag with a process ID and use physically tagged caches.
- 2023-24 Q3(b): 1 fault(13), 0 PT hit, 3 TLB hit, 8 fault(14), 11 PT hit, 3 TLB hit, 12 fault(15).

- 16-entry TLB × 4 KB = 64 KB reach ≪ 2 MB → thrashing → larger pages or a larger / two-level TLB.

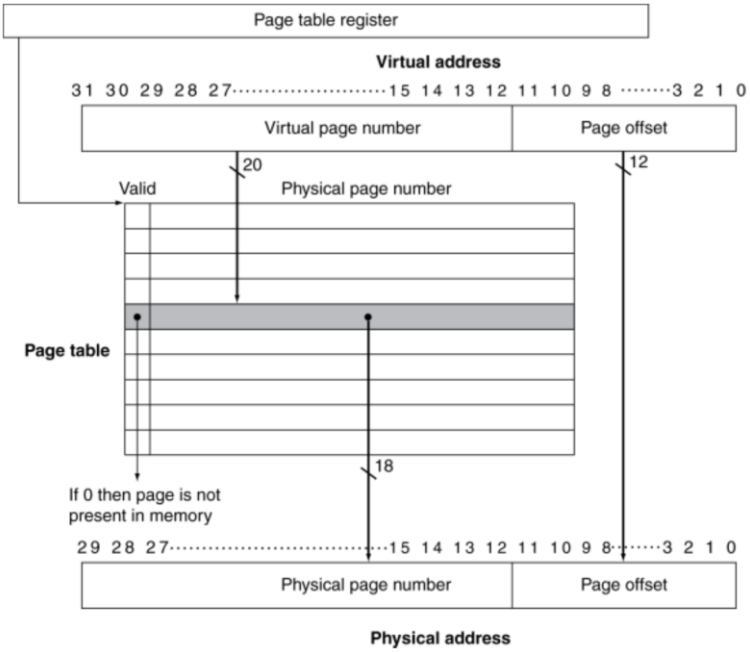
L28 · Coherence

- Problem: stale copies after a write. Coherence = reads return the most recent write; writes to one location seen in the same order by all.
- **Snooping** (bus, small systems): broadcast invalidate on write; other copies invalidated; later read misses and the owner supplies data. **Directory** (large systems): directory tracks sharers.
- Consistency: a write completes when all have seen it; writes are not reordered.

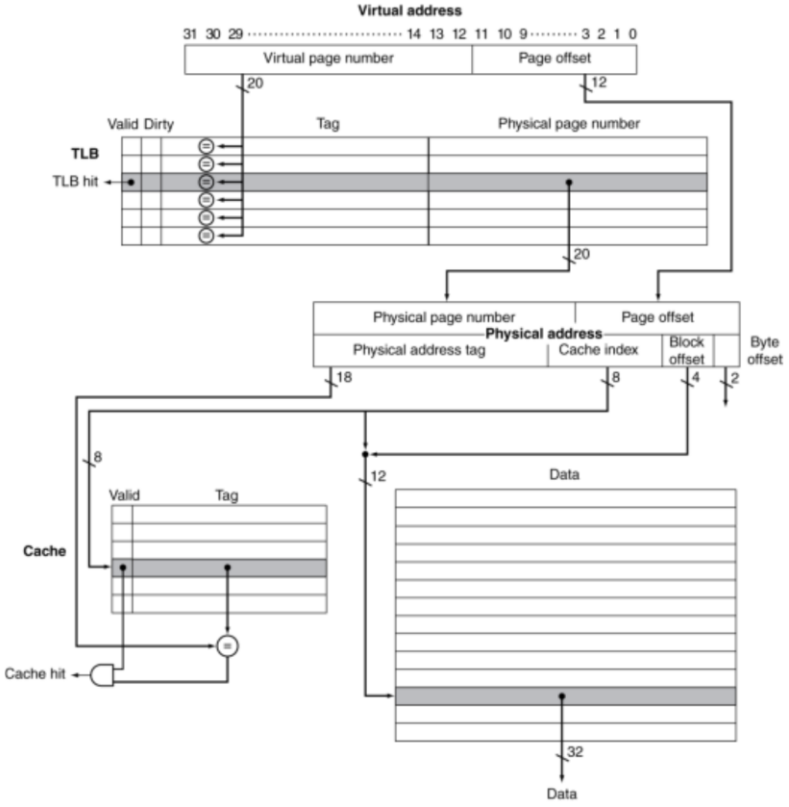
Key Diagrams to Reproduce



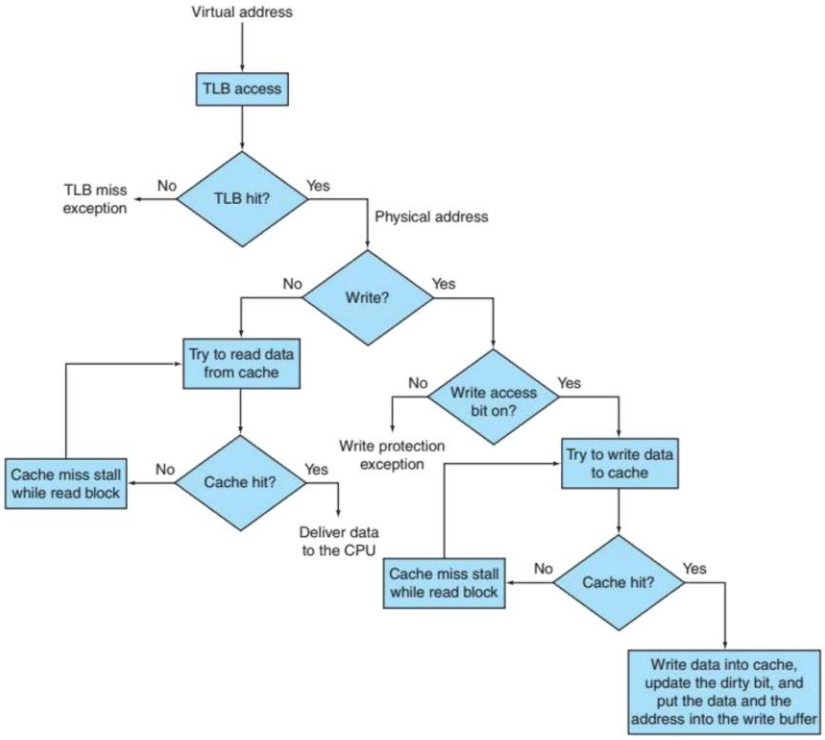
4-way set-associative cache organization (slide 44) — same as Fig. 21.1



Translation using a page table (slide 60) — same as Fig. 26.1



TLB and cache interaction (slide 69) — same as Fig. 27.2



TLB–cache read/write flowchart (slide 70) — same as Fig. 27.3